Stellar Photometric Variability: Gaia DR3 pilot

Query date 2026-10-12, Gaia DR3. Purpose: check data pulls and distributions.

In [1]:
```
from astroquery.gaia import Gaia
import pandas as pd
from datetime import date

query = """
SELECT TOP 2000
    v.source_id, v.best_class_name, v.best_class_score,
    g.phot_g_mean_mag, g.bp_rp, g.parallax, g.parallax_over_error
FROM gaiadr3.vari_classifier_result AS v
JOIN gaiadr3.gaia_source AS g USING (source_id)
"""

job = Gaia.launch_job(query)
df = job.get_results().to_pandas()

print("Query date:", date.today(), "| Release: Gaia DR3")
print(df.shape)
```

```
The current archive release 3.15 contains two breaking changes. Please see https://www.cosmos.
esa.int/web/gaia/news#GACSrelease315 for details. In case of questions or feedback, please con
sult the Gaia helpdesk at https://www.cosmos.esa.int/web/gaia/gaia-helpdesk.
Query date: 2026-10-18 | Release: Gaia DR3
(2000, 7)
```

In [2]:
```
df.to_csv("../data/raw/gaia_dr3_pilot2000_raw.csv", index=False)
```

In [3]:
```
df.head(10)
```

Out[3]:

	source_id	best_class_name	best_class_score	phot_g_mean_mag	bp_rp	parallax	par
0	5902290121412687616	RS	0.107063	17.863487	1.336353	0.345689	
1	5902290160100584576	SOLAR_LIKE	0.409437	14.860000	1.163411	1.405491	
2	5902291568849900032	SOLAR_LIKE	0.367021	14.525234	1.136244	1.153373	
3	5902295756411393920	ECL	0.182797	18.144302	1.641121	0.272087	
4	5902305789455121664	ECL	0.610995	18.512594	1.138405	0.220197	
5	5902306858911774720	ECL	0.923449	19.345819	1.443110	0.350725	
6	5902314933452036608	ECL	0.918724	19.510265	1.505346	0.108283	
7	5902321942859822848	RS	0.515493	16.701969	1.635451	0.137085	
8	5902324343716420992	ECL	0.567754	18.483683	1.359337	0.126774	
9	5902327023775227648	SOLAR_LIKE	0.408365	14.968561	1.011042	1.023321	

2. Class counts and missing values

In [4]:
```
print(df["best_class_name"].value_counts())
print()
```

```
print(df.isna().mean().round(3))
```

```
best_class_name
ECL                          677
SOLAR_LIKE                   455
RS                           274
S                            227
LPV                          163
DSCT|GDOR|SXPHE              131
RR                            57
YSO                            7
ELL                            7
AGN                            1
ACV|CP|MCP|ROAM|ROAP|SXARI     1
Name: count, dtype: int64

source_id              0.000
best_class_name        0.000
best_class_score       0.000
phot_g_mean_mag        0.000
bp_rp                  0.004
parallax               0.105
parallax_over_error    0.105
dtype: float64
```

3. Removing AGN (not stars)

In [5]:
```
stars = df[df["best_class_name"] != "AGN"].copy()
print(len(stars))
print(stars["best_class_name"].value_counts().head(8))
print(stars["best_class_score"].describe().round(2))
```

```
1999
best_class_name
ECL               677
SOLAR_LIKE        455
RS                274
S                 227
LPV               163
DSCT|GDOR|SXPHE   131
RR                 57
YSO                 7
Name: count, dtype: int64
count    1999.00
mean        0.50
std         0.29
min         0.00
25%         0.26
50%         0.48
75%         0.75
max         1.00
Name: best_class_score, dtype: float64
```

4. Finding the amplitude table

In [6]:
```
tables = Gaia.load_tables(only_names=True)
vari = [t.name for t in tables if "vari" in t.name]
for name in vari:
    print(name)
```

```
INFO: Retrieving tables... [astroquery.utils.tap.core]
INFO: Parsing tables... [astroquery.utils.tap.core]
INFO: Done. [astroquery.utils.tap.core]
gaiadr1.phot_variable_time_series_gfov
gaiadr1.phot_variable_time_series_gfov_statistical_parameters
gaiadr1.variable_summary
gaiadr2.vari_cepheid
gaiadr2.vari_classifier_class_definition
gaiadr2.vari_classifier_definition
gaiadr2.vari_classifier_result
gaiadr2.vari_long_period_variable
gaiadr2.vari_rotation_modulation
gaiadr2.vari_rrlyrae
gaiadr2.vari_short_timescale
gaiadr2.vari_time_series_statistics
gaiadr3.vari_spurious_signals
gaiadr3.vari_agn
gaiadr3.vari_cepheid
gaiadr3.vari_classifier_class_definition
gaiadr3.vari_classifier_definition
gaiadr3.vari_classifier_result
gaiadr3.vari_compact_companion
gaiadr3.vari_eclipsing_binary
gaiadr3.vari_epoch_radial_velocity
gaiadr3.vari_long_period_variable
gaiadr3.vari_microlensing
gaiadr3.vari_ms_oscillator
gaiadr3.vari_planetary_transit
gaiadr3.vari_planetary_transit_13june2022
gaiadr3.vari_rad_vel_statistics
gaiadr3.vari_rotation_modulation
gaiadr3.vari_rrlyrae
gaiadr3.vari_short_timescale
gaiadr3.vari_summary
gaiafpr.vari_epoch_radial_velocity
gaiafpr.vari_long_period_variable
gaiafpr.vari_rad_vel_statistics
```

In [7]:
```
t = Gaia.load_table("gaiadr3.vari_summary")
for col in t.columns:
    if "g_fov" in col.name:
        print(col.name)
```

```
num_selected_g_fov
mean_obs_time_g_fov
time_duration_g_fov
min_mag_g_fov
max_mag_g_fov
mean_mag_g_fov
median_mag_g_fov
range_mag_g_fov
trimmed_range_mag_g_fov
std_dev_mag_g_fov
skewness_mag_g_fov
kurtosis_mag_g_fov
mad_mag_g_fov
abbe_mag_g_fov
iqr_mag_g_fov
stetson_mag_g_fov
std_dev_over_rms_err_mag_g_fov
outlier_median_g_fov
```

5. Amplitude from vari_summary (G band)

In [8]:
```
query = """
SELECT TOP 2000
    v.source_id, v.best_class_name, v.best_class_score,
    g.phot_g_mean_mag, g.bp_rp, g.parallax, g.parallax_over_error,
    s.num_selected_g_fov, s.mean_mag_g_fov, s.range_mag_g_fov,
    s.trimmed_range_mag_g_fov, s.std_dev_mag_g_fov
FROM gaiadr3.vari_classifier_result AS v
JOIN gaiadr3.gaia_source AS g USING (source_id)
JOIN gaiadr3.vari_summary AS s USING (source_id)
WHERE v.best_class_name <> 'AGN'
"""

job = Gaia.launch_job(query)
df2 = job.get_results().to_pandas()

print("Query date:", date.today(), "| Release: Gaia DR3")
print(df2.shape)
print(df2["best_class_name"].value_counts().head(8))
print(df2.isna().mean().round(3))
df2.to_csv("../data/raw/gaia_dr3_pilot_amplitude_raw.csv", index=False)
df2.head()
```

```
Query date: 2026-10-18 | Release: Gaia DR3
(2000, 12)
best_class_name
ECL                703
SOLAR_LIKE         424
RS                 267
S                  248
LPV                176
DSCT|GDOR|SXPHE    117
RR                  47
ELL                  8
Name: count, dtype: int64
source_id                  0.000
best_class_name            0.000
best_class_score           0.000
phot_g_mean_mag            0.000
bp_rp                      0.004
parallax                   0.116
parallax_over_error        0.116
num_selected_g_fov         0.000
mean_mag_g_fov             0.000
range_mag_g_fov            0.000
trimmed_range_mag_g_fov    0.000
std_dev_mag_g_fov          0.000
dtype: float64
```

Out[8]:

	source_id	best_class_name	best_class_score	phot_g_mean_mag	bp_rp	parallax	pa
0	5902284967451738240	ECL	0.128291	18.303106	1.571106	-0.043814	
1	5902304041435463808	SOLAR_LIKE	0.254027	16.107353	1.143901	0.830458	
2	5902310909054835200	S	0.577000	18.085020	1.386198	NaN	
3	5902317613537088256	ECL	0.850763	16.465290	1.103224	0.403926	
4	5902317785331312640	ECL	0.785340	16.832952	1.368202	0.531456	

6. Finding period columns

In [9]:
```
tables_to_check = [
    "gaiadr3.vari_eclipsing_binary",
    "gaiadr3.vari_rrlyrae",
    "gaiadr3.vari_cepheid",
    "gaiadr3.vari_long_period_variable",
    "gaiadr3.vari_rotation_modulation",
    "gaiadr3.vari_short_timescale",
]

for name in tables_to_check:
    try:
        t = Gaia.load_table(name)
        cols = [c.name for c in t.columns
                if "period" in c.name or "frequency" in c.name or "amplitude" in c.name]
        print(name)
        print("   ", cols)
    except Exception as e:
        print(name, "-> failed:", str(e)[:80])
```

```
gaiadr3.vari_eclipsing_binary
    ['frequency', 'frequency_error', 'geom_model_cosine_half_period_amplitude', 'geom_model_co
sine_half_period_amplitude_error', 'geom_model_cosine_half_period_phase', 'geom_model_cosine_h
alf_period_phase_error']
gaiadr3.vari_rrlyrae
    []
gaiadr3.vari_cepheid
    []
gaiadr3.vari_long_period_variable
    ['frequency', 'frequency_error', 'amplitude']
gaiadr3.vari_rotation_modulation
    ['segments_rotation_period', 'segments_rotation_period_error', 'segments_rotation_period_f
ap', 'best_rotation_period', 'best_rotation_period_error']
gaiadr3.vari_short_timescale
    ['amplitude_estimate', 'frequency']
```

In [10]:
```
for name in ["gaiadr3.vari_rrlyrae", "gaiadr3.vari_cepheid"]:
    t = Gaia.load_table(name)
    print(name)
    print([c.name for c in t.columns])
    print()
```

```
gaiadr3.vari_rrlyrae
['solution_id', 'source_id', 'pf', 'pf_error', 'p1_o', 'p1_o_error', 'epoch_g', 'epoch_g_erro
r', 'epoch_bp', 'epoch_bp_error', 'epoch_rp', 'epoch_rp_error', 'epoch_rv', 'epoch_rv_error',
'int_average_g', 'int_average_g_error', 'int_average_bp', 'int_average_bp_error', 'int_average
_rp', 'int_average_rp_error', 'average_rv', 'average_rv_error', 'peak_to_peak_g', 'peak_to_pea
k_g_error', 'peak_to_peak_bp', 'peak_to_peak_bp_error', 'peak_to_peak_rp', 'peak_to_peak_rp_er
ror', 'peak_to_peak_rv', 'peak_to_peak_rv_error', 'metallicity', 'metallicity_error', 'r21_g',
'r21_g_error', 'r31_g', 'r31_g_error', 'phi21_g', 'phi21_g_error', 'phi31_g', 'phi31_g_error',
'num_clean_epochs_g', 'num_clean_epochs_bp', 'num_clean_epochs_rp', 'num_clean_epochs_rv', 'zp
_mag_g', 'zp_mag_bp', 'zp_mag_rp', 'num_harmonics_for_p1_g', 'num_harmonics_for_p1_bp', 'num_h
armonics_for_p1_rp', 'num_harmonics_for_p1_rv', 'reference_time_g', 'reference_time_bp', 'refe
rence_time_rp', 'reference_time_rv', 'fund_freq1', 'fund_freq1_error', 'fund_freq2', 'fund_fre
q2_error', 'fund_freq1_harmonic_ampl_g', 'fund_freq1_harmonic_ampl_g_error', 'fund_freq1_harmo
nic_phase_g', 'fund_freq1_harmonic_phase_g_error', 'fund_freq1_harmonic_ampl_bp', 'fund_freq1_
harmonic_ampl_bp_error', 'fund_freq1_harmonic_phase_bp', 'fund_freq1_harmonic_phase_bp_error',
'fund_freq1_harmonic_ampl_rp', 'fund_freq1_harmonic_ampl_rp_error', 'fund_freq1_harmonic_phase
_rp', 'fund_freq1_harmonic_phase_rp_error', 'fund_freq1_harmonic_ampl_rv', 'fund_freq1_harmoni
c_ampl_rv_error', 'fund_freq1_harmonic_phase_rv', 'fund_freq1_harmonic_phase_rv_error', 'best_
classification', 'g_absorption', 'g_absorption_error']

gaiadr3.vari_cepheid
['solution_id', 'source_id', 'pf', 'pf_error', 'p1_o', 'p1_o_error', 'epoch_g', 'epoch_g_erro
r', 'epoch_bp', 'epoch_bp_error', 'epoch_rp', 'epoch_rp_error', 'epoch_rv', 'epoch_rv_error',
'int_average_g', 'int_average_g_error', 'int_average_bp', 'int_average_bp_error', 'int_average
_rp', 'int_average_rp_error', 'average_rv', 'average_rv_error', 'peak_to_peak_g', 'peak_to_pea
k_g_error', 'peak_to_peak_bp', 'peak_to_peak_bp_error', 'peak_to_peak_rp', 'peak_to_peak_rp_er
ror', 'peak_to_peak_rv', 'peak_to_peak_rv_error', 'metallicity', 'metallicity_error', 'r21_g',
'r21_g_error', 'r31_g', 'r31_g_error', 'phi21_g', 'phi21_g_error', 'phi31_g', 'phi31_g_error',
'num_clean_epochs_g', 'num_clean_epochs_bp', 'num_clean_epochs_rp', 'num_clean_epochs_rv', 'zp
_mag_g', 'zp_mag_bp', 'zp_mag_rp', 'num_harmonics_for_p1_g', 'num_harmonics_for_p1_bp', 'num_h
armonics_for_p1_rp', 'num_harmonics_for_p1_rv', 'reference_time_g', 'reference_time_bp', 'refe
rence_time_rp', 'reference_time_rv', 'fund_freq1', 'fund_freq1_error', 'fund_freq2', 'fund_fre
q2_error', 'fund_freq1_harmonic_ampl_g', 'fund_freq1_harmonic_ampl_g_error', 'fund_freq1_harmo
nic_phase_g', 'fund_freq1_harmonic_phase_g_error', 'fund_freq1_harmonic_ampl_bp', 'fund_freq1_
harmonic_ampl_bp_error', 'fund_freq1_harmonic_phase_bp', 'fund_freq1_harmonic_phase_bp_error',
'fund_freq1_harmonic_ampl_rp', 'fund_freq1_harmonic_ampl_rp_error', 'fund_freq1_harmonic_phase
_rp', 'fund_freq1_harmonic_phase_rp_error', 'fund_freq1_harmonic_ampl_rv', 'fund_freq1_harmoni
c_ampl_rv_error', 'fund_freq1_harmonic_phase_rv', 'fund_freq1_harmonic_phase_rv_error', 'p2_
o', 'p2_o_error', 'type_best_classification', 'type2_best_sub_classification', 'mode_best_clas
sification', 'multi_mode_best_classification']
```

7. RR Lyrae and Cepheid pull

In [11]:

```
q_rr = """
SELECT TOP 2000
    r.source_id, r.pf, r.p1_o, r.peak_to_peak_g, r.int_average_g,
    r.num_clean_epochs_g, r.best_classification,
    g.bp_rp, g.parallax, g.parallax_over_error
FROM gaiadr3.vari_rrlyrae AS r
JOIN gaiadr3.gaia_source AS g USING (source_id)
"""

q_cep = """
SELECT TOP 2000
    c.source_id, c.pf, c.p1_o, c.peak_to_peak_g, c.int_average_g,
    c.num_clean_epochs_g, c.type_best_classification, c.mode_best_classification,
    g.bp_rp, g.parallax, g.parallax_over_error
FROM gaiadr3.vari_cepheid AS c
JOIN gaiadr3.gaia_source AS g USING (source_id)
"""
```

```
rr = Gaia.launch_job(q_rr).get_results().to_pandas()
cep = Gaia.launch_job(q_cep).get_results().to_pandas()

print("Query date:", date.today(), "| Release: Gaia DR3")
print("RR Lyrae:", rr.shape, "| Cepheids:", cep.shape)

rr.to_csv("../data/raw/gaia_dr3_rrlyrae_pilot_raw.csv", index=False)
cep.to_csv("../data/raw/gaia_dr3_cepheid_pilot_raw.csv", index=False)

print(rr.isna().mean().round(3))
print(cep.isna().mean().round(3))
```

```
Query date: 2026-10-18 | Release: Gaia DR3
RR Lyrae: (2000, 10) | Cepheids: (2000, 11)
source_id              0.000
pf                     0.378
p1_o                   0.618
peak_to_peak_g         0.000
int_average_g          0.000
num_clean_epochs_g     0.000
best_classification    0.000
bp_rp                  0.081
parallax               0.026
parallax_over_error    0.026
dtype: float64
source_id                   0.000
pf                          0.392
p1_o                        0.610
peak_to_peak_g              0.000
int_average_g               0.000
num_clean_epochs_g          0.000
type_best_classification    0.000
mode_best_classification    0.000
bp_rp                       0.000
parallax                    0.000
parallax_over_error         0.000
dtype: float64
```

8. Combined period column

In [12]:

```
import numpy as np
import matplotlib.pyplot as plt

for d in (rr, cep):
    d["period"] = d["pf"].fillna(d["p1_o"])
    d["period_type"] = np.where(d["pf"].notna(), "fundamental", "overtone")

print(rr["period"].isna().sum(), cep["period"].isna().sum(), "stars with no period")
print(rr["period_type"].value_counts())
print(cep["period_type"].value_counts())
```

```
0 11 stars with no period
period_type
fundamental    1245
overtone        755
Name: count, dtype: int64
period_type
fundamental    1216
overtone        784
Name: count, dtype: int64
```

9. First scatter plots

In [13]:

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
for a, d, title in [(ax[0], rr, "RR Lyrae"), (ax[1], cep, "Cepheids")]:
    d2 = d.dropna(subset=["period"])
    for k, grp in d2.groupby("period_type"):
        a.scatter(np.log10(grp["period"]), grp["peak_to_peak_g"], s=6, alpha=0.5, label=k)
    a.set_xlabel("log10 period (days)")
    a.set_ylabel("peak-to-peak G (mag)")
    a.set_title(title)
    a.legend()
plt.tight_layout()
plt.show()
```

